Project – Low Level Design

on

IaC Provisioning for Non-Profit Support System

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Submitted by:

Sr no	Student Name	Enrolment Number
1.	Ananya Subramanya Rao	EN22CS301120
2.	Anirudh Vyas	EN22CS301131
3.	Anushka Kashyap	EN22CS301180
4.	Atharv Chaturvedi	EN22CS301228
5.	Harshit Panchal	EN23CS3L1010

Group Name: Group 03D2

Project Number: DO-15

Industry Mentor Name: Vaibhav Sir

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2026

Table of Contents

1. **Introduction**
   1.1. Scope of the document
   1.2. Intended Audience
   1.3. System overview

2. **System Design**
   2.1. Application Design
   2.2. Process Flow
   2.3. Information Flow
   2.4. Components Design
   2.5. Docker Implementation
   2.6. CI/CD Pipeline
   2.7. Infrastructure as Code (Terraform)
   2.8. Kubernetes Configuration
   2.9. Key Design Considerations
   API Catalogue

3. **References**

1. Introduction

This Low-Level Design (LLD) document provides a technical description of the infrastructure, architecture, and implementation decisions made for the IaC Provisioning for Non-Profit Support System, So our web app name is SupportHub — an AI-Powered Microservices Platform for Non-Profit Support Operations system.

SupportHub is a production-grade, microservices-based DevOps platform that automates the provisioning, containerization, and deployment of two AI-powered non-profit tools — TriageAgent (an AI communication triage system) and QuizBot (an AI-driven staff training platform) — as a unified microservices platform. The platform uses Infrastructure as Code (IaC) tools, container orchestration, and CI/CD pipelines to eliminate all manual setup and enable repeatable, one-command deployment of the entire system. Built using Docker for containerization, managed through an Nginx as an API gateway, GitHub Actions for CI/CD automation, Terraform for infrastructure provisioning, and Kubernetes for container orchestration.

Problem Statement

Automate the provisioning of infrastructure for a Non-Profit system using Infrastructure as Code (IaC) tools like Terraform or Ansible. The project involves script-based deployment of local Docker environments or cloud instances (e.g., AWS), including the automated installation of core dependencies such as Terraform or Ansible. This approach eliminates manual configuration errors and enables repeatable, scalable environment setup.

Project Objectives

- **Full Containerization**: Package all 5 services as Docker containers connected through a shared supporthub-network Docker bridge network.
- **Infrastructure as Code**: Provision the entire platform using Terraform, it scripts provision and deploy the entire platform with a single command.
- **Automated Dependency Installation**: install.sh bootstraps Docker, Terraform, and kubectl on any fresh Linux machine automatically.
- **CI/CD Pipeline**: GitHub Actions builds all 5 Docker images, runs tests, builds images and pushes to Docker Hub, and deploys on every push to main.
- **Kubernetes Orchestration**: K8s manages container lifecycle with Deployments, Services, and Ingress for auto-restart, health checks, and horizontal scaling.
- **Nginx API Gateway**: Nginx routes all external traffic to the correct microservice based on URL path prefix.
- **Repeatable & Scalable**: One terraform apply command provisions the entire SupportHub platform identically on any machine, every time.

1.1 Scope of the document

This LLD document provides a detailed technical specification for the SupportHub DevOps platform. The document covers:

- The containerization strategy for all five services using Docker
- The infrastructure provisioning approach using Terraform
- The container orchestration design using Kubernetes
- The automated CI/CD pipeline design using GitHub Actions
- The API Gateway configuration using Nginx
- The network architecture and inter-service communication
- Complete technology stack with version details and justification
- All configuration files, scripts, and deployment artifacts created for the project

This document does not cover the internal application logic of TriageAgent or QuizBot, as those were developed in prior Datagami/IBM courses. The scope is strictly the DevOps infrastructure layer built around those applications.

1.2 Intended Audience

Audience	Relevance
Project Evaluators	Assess technical depth and correctness of the DevOps implementation
Team Members (Group 03D2)	Reference guide for every design decision made during the project
DevOps Engineers / Developers	Maintain Docker containers, Terraform scripts, GitHub Actions workflows, and K8s configs
Non-Profit Organization Staff	End users who benefit from reliable, always-on services maintained by the automated pipeline

Readers are expected to have basic familiarity with containerization concepts, cloud infrastructure, and software deployment practices.

1.3 System overview

SupportHub is a containerized microservices platform that automates the deployment and management of two AI-powered non-profit applications. **TriageAgent** automatically classifies incoming messages by urgency and intent, extracts named entities using spaCy NER, generates professional draft responses using Google Gemini 2.5 Flash via LangChain, and routes messages to the correct department — all in under 10 seconds, with 16 REST API endpoints and Firebase Firestore for real-time data. **QuizBot** parses donor emails, builds vector embeddings using Sentence Transformers stored in ChromaDB, and generates contextual MCQ quizzes using a RAG pipeline powered by Gemini AI, tracking learning progress over time with SQLite persistence. The system runs all components simultaneously with a single docker compose up -d command — making it fully operational in under 2 minutes from a fresh clone.

Component	Technology	Purpose
Containerization	Docker	Packages each service into isolated containers
Local Orchestration	Docker Compose	Runs all 5 containers together locally

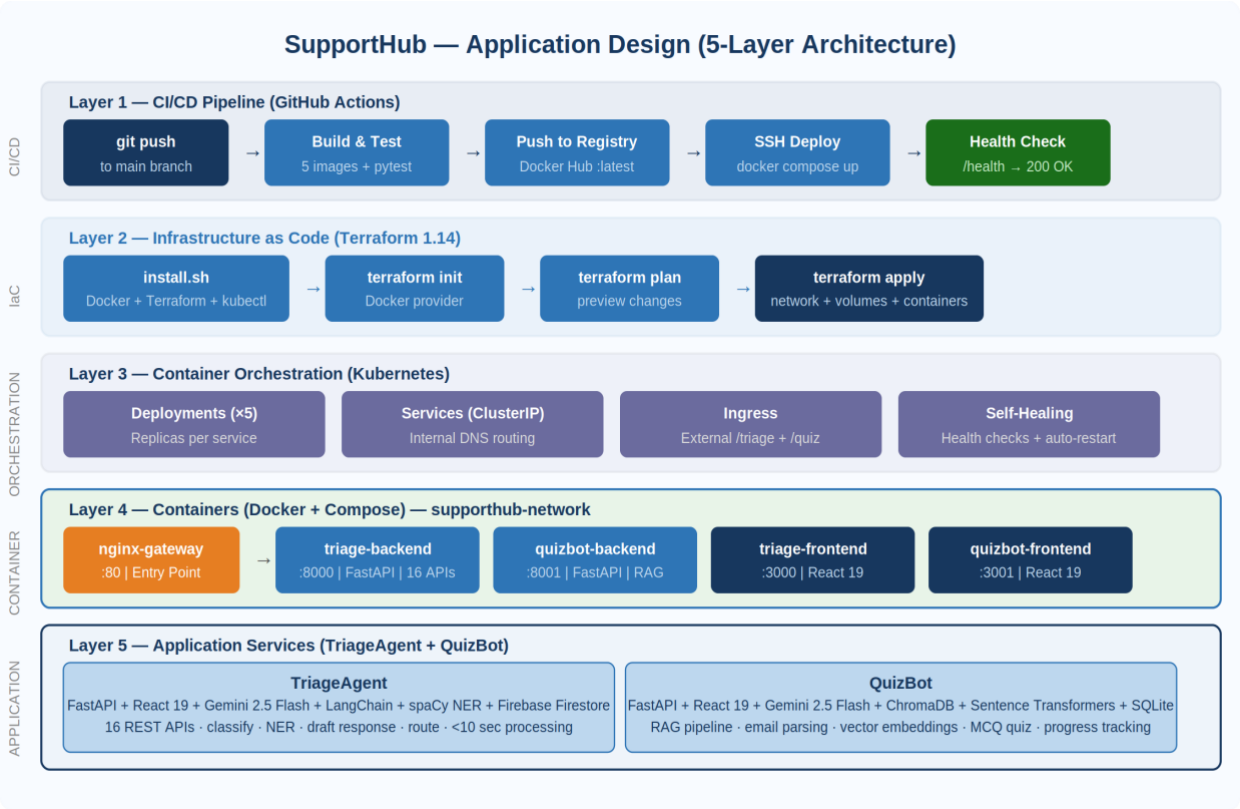
Production Orchestration	Kubernetes	Manages containers at scale in production
Infrastructure as Code	Terraform	Automates provisioning with zero manual steps
CI/CD Pipeline	GitHub Actions	Auto-tests, builds, and deploys on every push
Nginx API Gateway	nginx:alpine + config	Routes all traffic to correct services
AI Backend	FastAPI + Python 3.11 + LangChain + DB	REST APIs for both TriageAgent and QuizBot
Frontend	React 19 + Vite 7	User interfaces for both applications

The platform processes requests end-to-end: a user opens http://localhost/ and sees the SupportHub landing page. Clicking 'Launch TriageAgent' routes to http://localhost/triage/ where they can submit messages for AI triage. Clicking 'Launch QuizBot' routes to http://localhost/quiz/ where staff can generate and take quizzes. All routing is handled by Nginx without the user being aware of the underlying container architecture.

2. System Design

2.1 Application Design

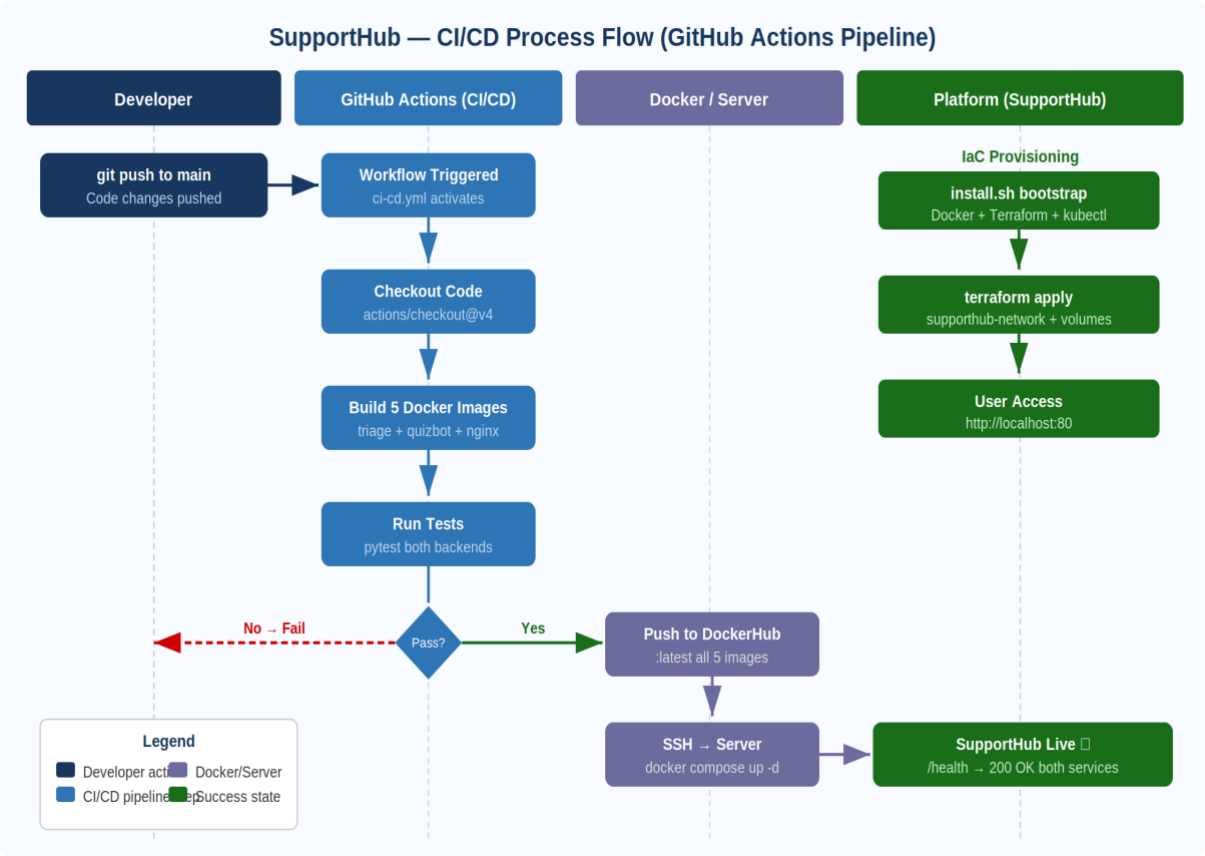
SupportHub follows a **5-layer containerized microservices architecture** with IaC provisioning, Nginx API gateway, CI/CD automation, and Kubernetes orchestration layered on top. Each layer has a single responsibility and clean interfaces — no layer bypasses another.



2.2 Process Flow

SupportHub has two distinct process flows: the **CI/CD Pipeline Flow** (triggered automatically on every code push) and the **IaC Provisioning Flow** (run once to set up infrastructure from scratch on any new machine).

CI/CD Pipeline Process flow (GitHub Actions)



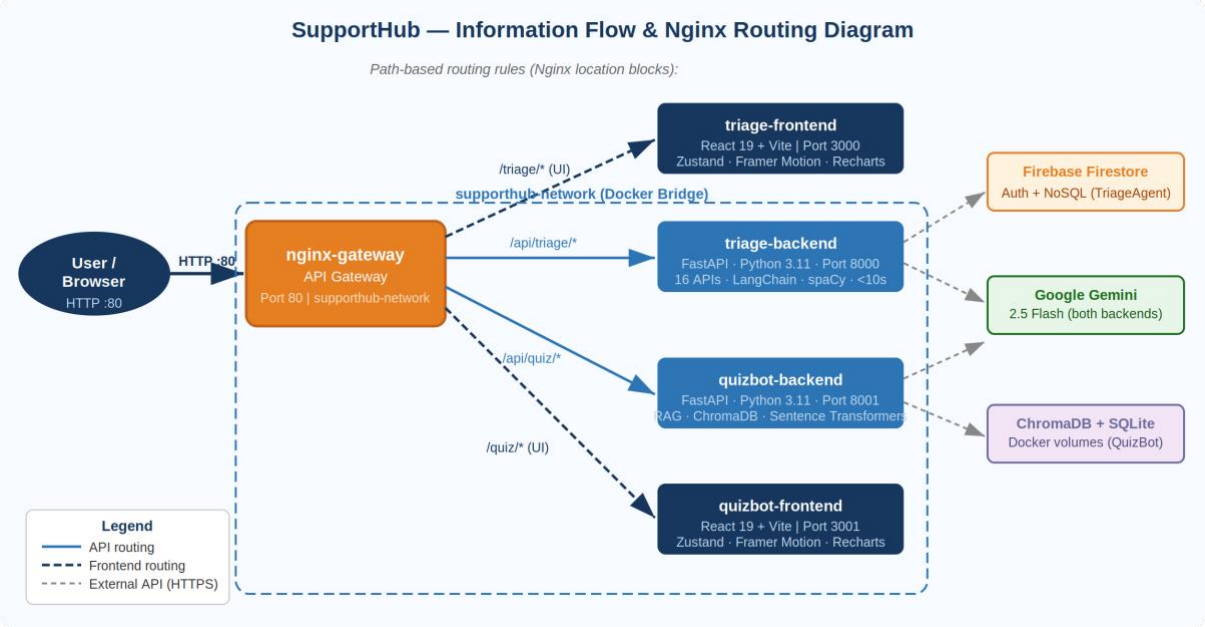
IaC Provisioning Flow (Terraform 1.14)

Step	Command	What Happens
1	**./scripts/install.sh**	Auto-installs Docker, Docker Compose, Terraform 1.14, and kubectl on any fresh Linux machine
2	**terraform init**	Initializes Terraform workspace, downloads kreuzwerker/docker provider
3	**terraform plan**	Previews all resources to be created without making any changes to the system
4	**terraform apply**	Creates supporthub-network (bridge), Docker volumes (ChromaDB, SQLite), and provisions all 5 containers
5	**docker compose up -d**	Starts all services in detached mode; Nginx becomes the public entry point on port 80
6	**Platform live**	All 5 containers healthy, CI/CD pipeline connected, SupportHub ready for users

2.3 Information Flow

2.3.1 Data Flow - Nginx Request Routing

All user requests enter SupportHub through the Nginx API gateway on port 80. Nginx uses **upstream blocks and location-based rewrite rules** to route traffic. The SupportHub landing page (portal/index.html) is served at the root. Backends never receive external paths directly — Nginx rewrites and proxies them internally.

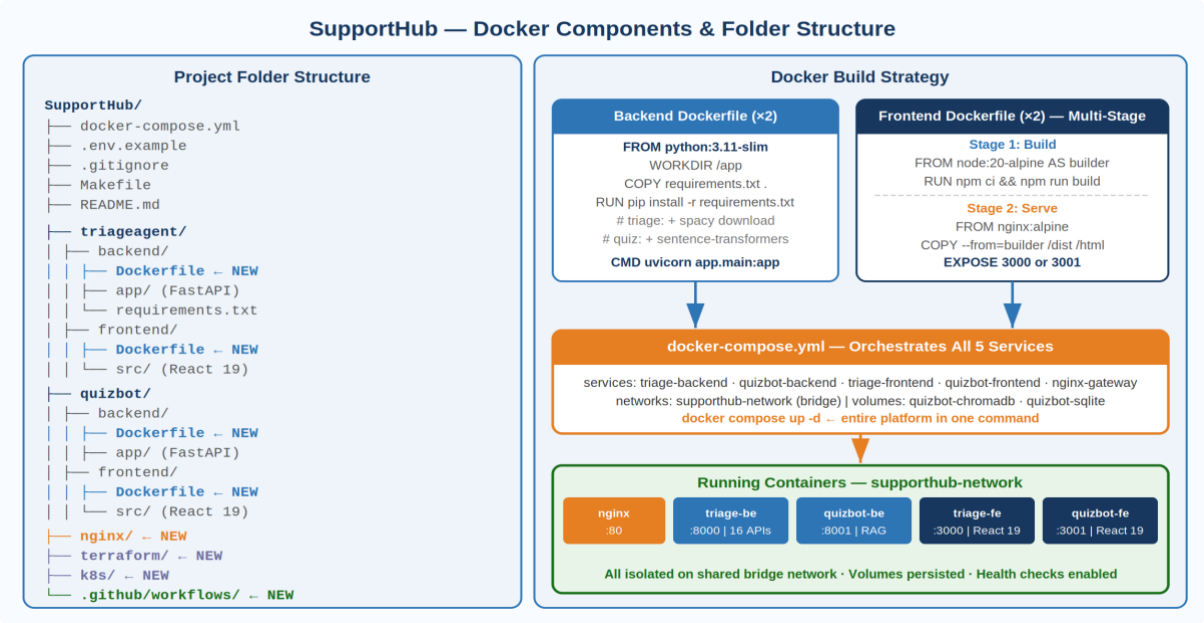


2.3.2 Container Network Design

Container	Internal Host	External URL Path	External Port	Connected To
supporthub-nginx	nginx	/ (root)	0.0.0.0:80	All 4 service containers
triage-backend	triage-backend	/triage/api/	0.0.0.0:8000	Firebase, Gemini API, spaCy
triage-frontend	triage-frontend	/triage/	0.0.0.0:3000	Nginx (routed via proxy)
quiz-backend	quiz-backend	/quiz/api	0.0.0.0:8001	Firebase, Gemini API, ChromaDB
quiz-frontend	quiz-frontend	/quiz/	0.0.0.0:3001	Nginx (routed via proxy)

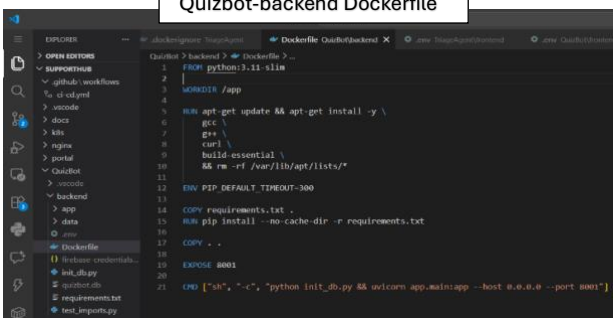
2.4 Docker Components Design

Each of the 5 services has a dedicated Dockerfile. Backend services use a **single-stage build** (python:3.11-slim). Frontend services use **multi-stage builds** — Stage 1: node:18-alpine to build the React app; Stage 2: nginx:alpine to serve the compiled static files. The Nginx container additionally serves the portal/index.html SupportHub landing page.

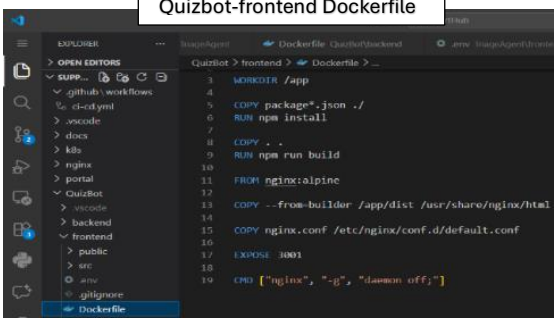


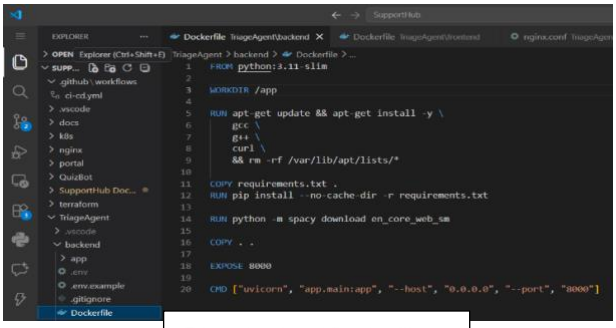
2.5 Docker Implementation

Quizbot-backend Dockerfile

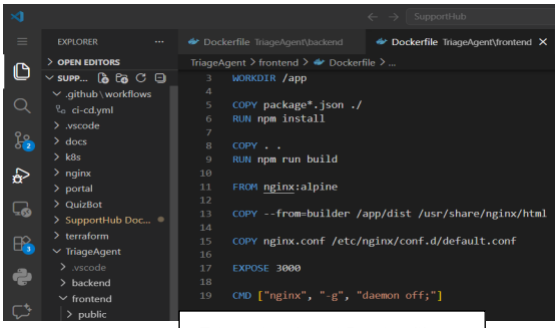


Quizbot-frontend Dockerfile



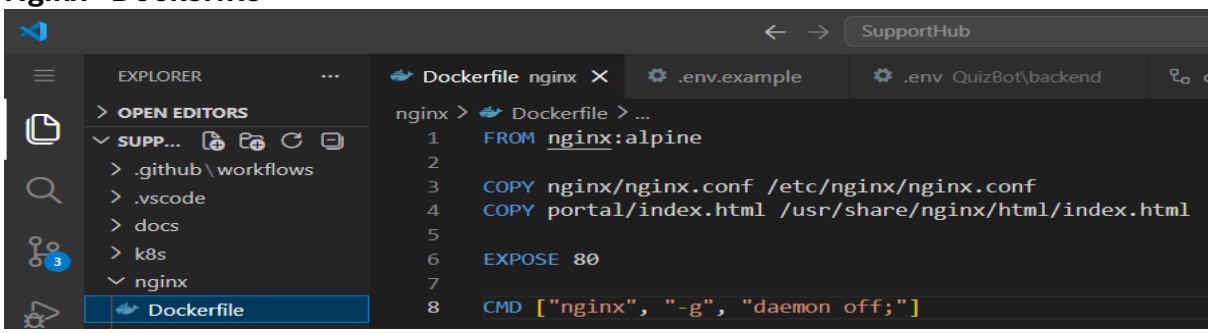


Triage-backend Dockerfile



Triage-backend Dockerfile

Nginx - Dockerfile



2.5.1 Docker Compose – Service Orchestration

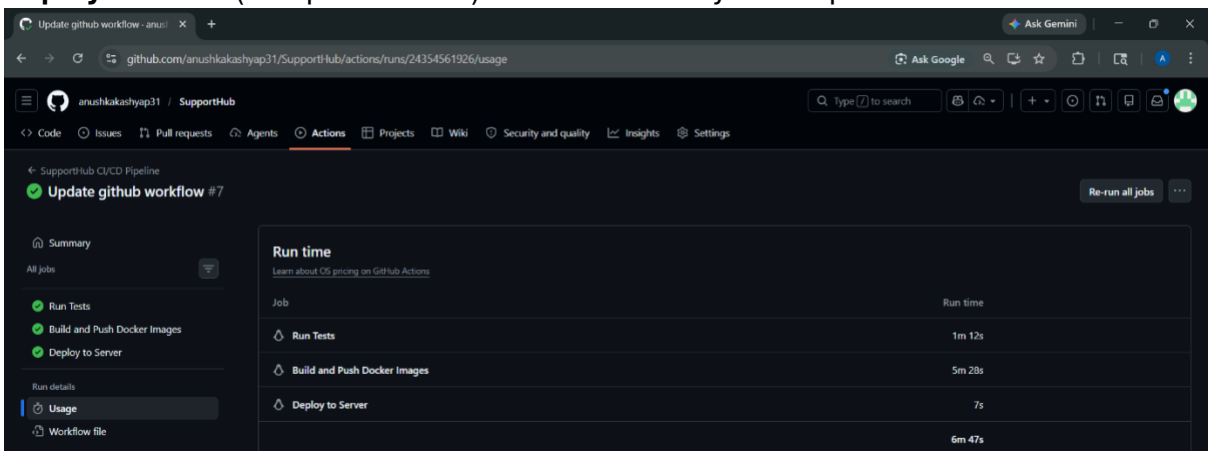
```yaml
services:
  nginx:
    build:
      context: .
      dockerfile: nginx/Dockerfile
    container_name: supporthub-nginx
    ports:
      - "80:80"
    depends_on:
      - triage-frontend
      - triage-backend
      - quiz-frontend
      - quiz-backend
    restart: unless-stopped
    networks:
      - supporthub-network

networks:
  supporthub-network:
    driver: bridge
```

2.5.2 Nginx Configuartion – API Gateway

```nginx
http {
    server {
        listen 80;

        location /triage/api/ {
            rewrite ^/triage/api/(.*) /api/$1 break;
            proxy_pass http://triage_backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        location /triage/ {
            rewrite ^/triage/(.*) /$1 break;
            proxy_pass http://triage_frontend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        location /quiz/api/ {
            rewrite ^/quiz/api/(.*) /api/$1 break;
            proxy_pass http://quiz_backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        location /quiz/ {
            rewrite ^/quiz/(.*) /$1 break;
            proxy_pass http://quiz_frontend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        location / {
            root /usr/share/nginx/html;
            index index.html;
            try_files $uri $uri/ /index.html;
        }
```

2.6 CI/CD Pipeline Design – GitHub Actions

The SupportHub CI/CD pipeline in **.github/workflows/ci-cd.yml** uses GitHub Actions with 3 sequential jobs. Every push to the main branch triggers an automated workflow that **tests** (import validation), **builds and push** (Docker images to DockerHub), and **deploy** to server (compose restart). Jobs 2 and 3 only run on pushes to main.



2.7 Terraform IaC Design

Terraform uses the **hashicorp/local** and **hashicorp/null** providers — no cloud account required. It provisions the local environment end-to-end by orchestrating shell scripts, making it runnable on any developer machine or Linux server.

File	Purpose
terraform/main.tf	Core IaC — defines null_resource provisioners that run install.sh and deploy.sh
terraform/variables.tf	Input variables: project_name, environment, port numbers with validation
terraform/outputs.tf	Output values: all service URLs after provisioning
terraform/scripts/install.sh	Bash script: auto-installs Docker, Terraform, kubectl on any Linux server
terraform/scripts/deploy.sh	Bash script: builds images, checks .env files, runs docker compose up

2.8 Kubernetes Configuration

File	Kind	Purpose
triage-backend-deployment.yml	Deployment	Runs triage-backend pod with health probes and resource limits
triage-backend-service.yml	Service (ClusterIP)	Internal networking on port 8000
triage-frontend-deployment.yml	Deployment	Runs triage-frontend pod
triage-frontend-service.yml	Service (ClusterIP)	Internal networking on port 3000
quizbot-backend-deployment.yml	Deployment	Runs quiz-backend with 512Mi memory request
quizbot-backend-service.yml	Service (ClusterIP)	Internal networking on port 8001
quizbot-frontend-deployment.yml	Deployment	Runs quiz-frontend pod
quizbot-frontend-service.yml	Service (ClusterIP)	Internal networking on port 3001
nginx-deployment.yml	Deployment	Runs Nginx API Gateway pod
ingress.yml	Ingress	External access with path-based routing to all services

2.9 Key Design Considerations

No.	Design Consideration	Description	Why This Choice Was Made
1	**Microservices Architecture**	TriageAgent and QuizBot deployed as independent services, each with own containers, communicating through Nginx.	Each service scales and fails independently. Two distinct AI pipelines stay isolated. Follows real-world industry patterns (Netflix, Zomato, Swiggy).
2	**Multi-Stage Docker Builds**	Frontend images use node:18-alpine to build React apps, then nginx:alpine to serve compiled static files. Each frontend also has its own nginx.conf.	Reduces final image size by 60–70%. Production images contain only compiled static files — no Node.js, no source code. Faster pulls and smaller attack surface.
3	**GitHub Actions over Jenkins**	CI/CD pipeline defined as YAML workflow files version-controlled inside the SupportHub repository.	No separate Jenkins server to provision or maintain. Workflow files live alongside code. Cloud-native, free tier sufficient for this project.
4	**Terraform Local Provider**	Terraform uses hashicorp/local and hashicorp/null providers — no cloud account required.	Maximally accessible for evaluation. Any evaluator runs terraform apply on any Linux machine and gets the full

		Scripts provision the environment locally via shell.	SupportHub platform running. Directly fulfills DO-15 IaC requirement.
5	**Nginx as Single Entry Point + Landing Page**	One Nginx container receives all traffic on port 80, serves the SupportHub landing page at /, and proxies /triage/* and /quiz/* with rewrite rules.	Single public port — backends never directly exposed. Path-based rewrites mean each app receives its original API paths without modification. Landing page is served at zero additional cost.
6	**Kubernetes Orchestration**	K8s Deployments manage lifecycle, replication, livenessProbes, and rolling updates for all services with defined resource limits.	Self-healing: crashed containers restart automatically. Rolling updates enable zero-downtime deploys. Resource limits prevent memory leaks from starving the host.
7	**Secrets via .env + GitHub Secrets**	API keys, Firebase credentials, and JWT secrets stored in .env files. Never hardcoded in Dockerfiles or app source code.	Hardcoded secrets in Docker images is a critical security vulnerability. .env files are gitignored. CI/CD uses encrypted GitHub repository Secrets for DOCKERHUB_USERNAME and DOCKERHUB_TOKEN.

2.10 API Catalogue

Base URL: http://localhost:80 | **Network:** supporthub-network | **Auth:** Authorization: Bearer {JWT}

Nginx Routing Configuration (nginx/nginx.conf): Nginx defines **upstream blocks** for each container by Docker service name, then uses **location blocks with rewrite rules** to strip the service prefix before proxying. This allows each app to continue using its original internal API paths unchanged.

Location Block	Proxy Pass Target	Rewrite / Purpose
/	root /usr/share/nginx/html (static)	Serves portal/index.html — SupportHub landing page
/triage/api/	http://triage_backend (upstream: triage-backend:8000)	Rewrite: /triage/api/(.*) → /api/$1. TriageAgent backend receives /api/* as normal.
/triage/	http://triage_frontend (upstream: triage-frontend:3000)	Rewrite: /triage/(.*) → /$1. TriageAgent React app served from root.
/quiz/api/	http://quiz_backend (upstream: quiz-backend:8001)	Rewrite: /quiz/api/(.*) → /api/$1. QuizBot backend receives /api/* as normal.
/quiz/	http://quiz_frontend (upstream: quiz-frontend:3001)	Rewrite: /quiz/(.*) → /$1. QuizBot React app served from root.

3. References

#	Reference	URL / Source
1	Docker Official Documentation	https://docs.docker.com/
2	Docker Compose Documentation	https://docs.docker.com/compose/
3	GitHub Actions Documentation	https://docs.github.com/en/actions
4	Terraform Official Documentation	https://developer.hashicorp.com/terraform/docs
5	Kubernetes Documentation	https://kubernetes.io/docs/
6	Nginx Official Documentation	https://nginx.org/en/docs/
7	TriageAgent GitHub Repository	https://github.com/anushkakashyap31/TriageAgent
8	QuizBot GitHub Repository	https://github.com/anushkakashyap31/QuizBot
9	SupportHub GitHub Repository	https://github.com/anushkakashyap31/SupportHub

Technology Stack — SupportHub DevOps Platform

Complete DevOps and application technology stack with versions, purpose, and justification.

DevOps / Infrastructure

#	Category	Technology	Version	Purpose	Justification
1	Containerization	**Docker**	v29.3.1	Package all services as portable, reproducible containers	Industry standard. Eliminates environment inconsistency. Runs identically on any OS.
2	Container Compose	**Docker Compose**	v5.1.1	Run all 5 services with one command	Declarative multi-service config. Dependency ordering. Volume and network management.
3	API Gateway	**Nginx**	Alpine	Route /triage/* and /quiz/* to correct microservice	Lightweight, high-performance reverse proxy. Path-based routing. CORS handled it.

4	IaC	**Terraform**	1.14.8	Provision all infrastructure as code	Declarative, plan-before-apply. kreuzwerker/docker provider. Fully repeatable provisioning.
5	Container Orchestration	**Kubernetes**	v1.34.1	Manage containers at scale with self-healing	Self-healing, rolling updates, horizontal scaling by replica count. Industry standard.
6	CI/CD	**GitHub Actions**	Latest	Automate build, test, push, deploy on every push	No server to maintain. Cloud-native. Workflow files version-controlled alongside code.
7	Backend base	**python:3.11-slim**	3.11	Minimal Python base for both backend containers	Slim: 3x smaller than full image. Compatible with LangChain, spaCy, ChromaDB.
8	Frontend build	**node:20-alpine**	20 LTS	Build React 19 apps inside container (multi-stage)	Minimal Alpine size. LTS stability. Used only in multi-stage build, not in final image.
9	Frontend serve	**nginx:alpine**	Alpine	Serve compiled React static files in production	Purpose-built for static serving. No Node.js overhead in production image.
10	Version Control GitHub Project Repository Link:	**Git + GoitHub**	v2.47.1	Source code versioning and collaboration	Industry standard. GitHub Actions CI/CD requires GitHub. Docker Hub secrets configured per repo.